

1/17/2025	9 AM - 10 AM
-----------	--------------

1. SQL Server: Basic Introduction

SQL Server is a relational database management system (RDBMS) developed by Microsoft. It is designed to store, retrieve, and manage data in a way that allows multiple users to access and manage the data concurrently. SQL Server is widely used in enterprise applications for handling large amounts of data, supporting online transaction processing (OLTP), and performing analytics.

Here's a breakdown of key concepts and components to help you understand the basics of SQL Server:

2. SQL Server Architecture

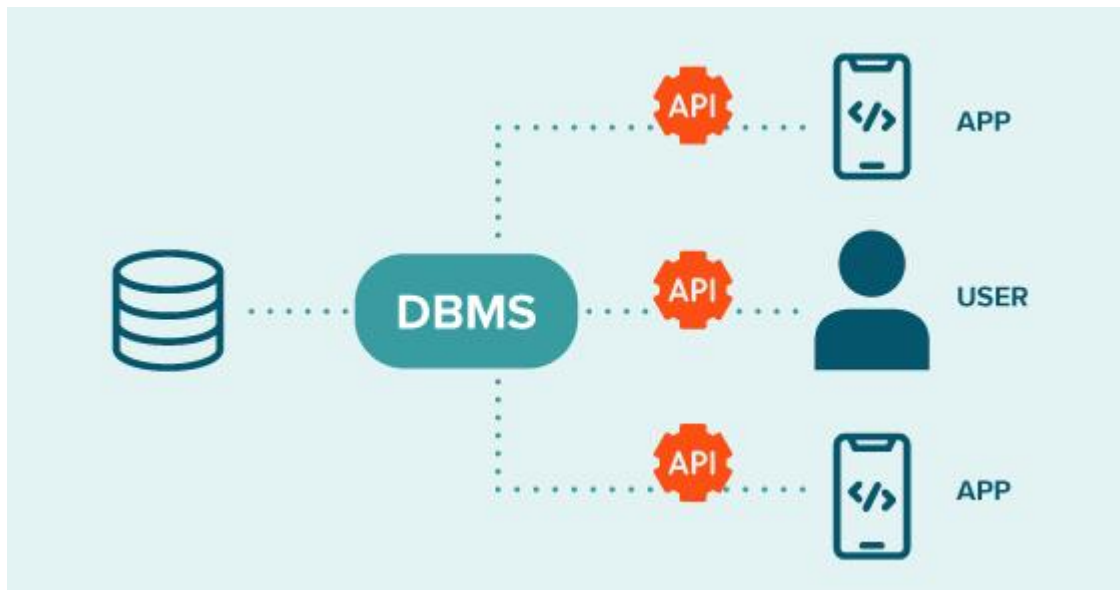
SQL Server operates based on a client-server architecture. It consists of two main parts:

- **Client:** The end-user application or tools (e.g., SQL Server Management Studio, application programs) that request data from SQL Server.
- **Server:** The SQL Server instance that processes client requests, manages the databases, and returns the requested data.

Key concepts in SQL Server architecture include:

- **Instance:** An instance of SQL Server is a running copy of the SQL Server Database Engine. Each instance can have its own databases, users, and system resources. Instance Location: C:\Program Files\Microsoft SQL Server\MSSQL16.MSSQLSERVER\MSSQL\Binn
- **Databases:** SQL Server stores data in databases, each of which contains tables, views, indexes, stored procedures, and other database objects.
- **Tables:** Data is stored in tables, which consist of rows and columns. Each table typically represents an entity (e.g., "Customers," "Orders").

Schemas: A schema is a collection of database objects that define the structure of a database, such as tables and views. Schemas help organize database objects.



ACID Properties:

🔍 **Atomicity:** If something goes wrong (e.g., Account 1 has insufficient balance), the entire transaction will be rolled back. Both updates (deducting and adding money) are treated as a single unit.

🔍 **Consistency:** The database constraints, such as ensuring the balance does not go negative, are respected. If an update violates the constraints, the transaction fails, and the database remains consistent.

🔍 **Isolation:** If another transaction tries to access or modify the accounts while this transaction is running, it will be isolated based on the isolation level. For instance, with Read Committed, no other transaction will be able to see the changes until this one is committed.

🔍 **Durability:** Once the transaction is committed, even if the system crashes immediately afterward, the changes (i.e., the \$100 deduction and addition) will not be lost and will persist when the system recovers.

Example:

```
CREATE TABLE Orders (
```

```
    OrderID INT,
```

```
    CustomerID INT,
```

```
    OrderAmount DECIMAL(10, 2) CHECK (OrderAmount >= 0)
```

```
);
```

```
insert into Orders(OrderID, CustomerID, OrderAmount) values (1,2,10)
```

Select * from Orders

insert into Orders(OrderID, CustomerID, OrderAmount) values (1,2,-2)

begin try

begin tran

insert into Orders(OrderID, CustomerID, OrderAmount) values (1,2,10)

insert into Orders(OrderID, CustomerID, OrderAmount) values (1,2,-2)

commit tran

end try

begin catch

PRINT 'Error Message: ' + ERROR_MESSAGE();

rollback tran

end catch

In one SQL instance:

begin tran

update Orders set OrderAmount=40 where OrderID=1

commit tran

In another instance:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED ; OR -- UNCOMMITTED

BEGIN TRANSACTION;

-- Transaction 1: Reading a record

SELECT * FROM Orders WHERE OrderID = 1;

-- This read is blocked if another transaction updates the record before committing.

COMMIT TRANSACTION;

1/17/2025	10 AM - 11 AM
-----------	---------------

1. SQL Server Components

SQL Server consists of several key components that work together to provide robust database management:

- **SQL Server Database Engine:** The core service for storing, processing, and securing data. It is responsible for query processing, transaction management, and data storage.
 - **SQL Server Management Studio (SSMS):** A tool for managing and interacting with SQL Server databases. It provides a graphical interface to query, manage, and administer databases.
 - **SQL Server Agent:** A job scheduling and automation tool that helps in automating tasks like backups, batch processes, and other maintenance tasks.
 - **SQL Server Reporting Services (SSRS):** A tool for creating, managing, and delivering reports from SQL Server data.
 - **SQL Server Integration Services (SSIS):** A data integration and ETL (Extract, Transform, Load) tool for moving and transforming data between systems.
 - **SQL Server Analysis Services (SSAS):** A tool for data analysis and OLAP (Online Analytical Processing) to help create data cubes and perform complex analysis.
-

1/17/2025	11 AM - 12 PM
-----------	---------------

3. SQL Server Databases and Data Types

SQL Server organizes data into **databases**. Each database contains multiple **tables**, which store the actual data in the form of rows and columns.

- **Tables:** Represent entities and hold data. Each table consists of columns (fields) and rows (records).
- **Data Types:** SQL Server supports a variety of data types for storing different kinds of data. Some common data types include:
 - INT, BIGINT – Integer types
 - VARCHAR, CHAR – String types
 - DATE, DATETIME, TIME – Date and time types
 - DECIMAL, FLOAT – Numeric types
 - BIT – Boolean type (0 or 1)

Summary of SQL Server Data Types

Category	Data Types
Numeric	INT , BIGINT , SMALLINT , TINYINT , DECIMAL , NUMERIC , FLOAT , REAL
String	CHAR , VARCHAR , TEXT , NCHAR , NVARCHAR , NTEXT , VARCHAR(MAX) , NVARCHAR(MAX)
Date/Time	DATE , TIME , DATETIME , DATETIME2 , SMALLDATETIME , DATETIMEOFFSET
Binary	BINARY , VARBINARY , IMAGE
Other	BIT , UNIQUEIDENTIFIER , MONEY , SMALLMONEY , XML , CURSOR , SQL_VARIANT , TABLE
Special	JSON , HIERARCHYID , GEOGRAPHY , GEOMETRY

In SQL Server, data types define the kind of data that can be stored in a column of a table or in a variable. SQL Server provides a wide range of data types for different types of data, such as numeric values, text, dates, and more.

Here's an overview of the commonly used SQL Server data types:

1. Numeric Data Types

These data types are used to store numeric values (integer or floating-point values).

- **INT**: Stores integer values from -2,147,483,648 to 2,147,483,647.
 - Example: INT
- **BIGINT**: Stores larger integers, from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
 - Example: BIGINT
- **SMALLINT**: Stores smaller integers, from -32,768 to 32,767.
 - Example: SMALLINT
- **TINYINT**: Stores very small integer values, from 0 to 255.
 - Example: TINYINT
- **DECIMAL(p, s)**: Stores fixed precision and scale numbers. p is the total number of digits, and s is the number of digits to the right of the decimal point.
 - Example: DECIMAL(10, 2) or DEC(10, 2)
- **NUMERIC(p, s)**: Functionally equivalent to DECIMAL. Stores fixed precision and scale numbers.
 - Example: NUMERIC(10, 2)
- **FLOAT**: Stores floating-point numbers with precision based on the system. FLOAT can store very large or very small numbers.
 - Example: FLOAT(53) (default precision 53)
- **REAL**: Stores floating-point numbers with less precision than FLOAT. Equivalent to FLOAT(24).
 - Example: REAL

2. String Data Types

These data types are used to store character or string values.

- **CHAR(n)**: Stores fixed-length strings. n specifies the number of characters (1 to 8,000).
 - Example: CHAR(10) stores exactly 10 characters.

- **VARCHAR(n)**: Stores variable-length strings with a maximum length of n characters (up to 8,000).
 - Example: VARCHAR(50) stores up to 50 characters.
- **TEXT**: Stores large variable-length text data. Deprecated, use VARCHAR(MAX) instead.
 - Example: TEXT
- **NCHAR(n)**: Stores fixed-length Unicode strings (each character takes 2 bytes). n specifies the number of characters (1 to 4,000).
 - Example: NCHAR(10) stores exactly 10 Unicode characters.
- **NVARCHAR(n)**: Stores variable-length Unicode strings with a maximum length of n characters (up to 4,000).
 - Example: NVARCHAR(100) stores up to 100 Unicode characters.
- **NTEXT**: Stores large variable-length Unicode text data. Deprecated, use NVARCHAR(MAX) instead.
 - Example: NTEXT
- **VARCHAR(MAX)**: Stores large variable-length strings (up to 2 GB).
 - Example: VARCHAR(MAX)
- **NVARCHAR(MAX)**: Stores large variable-length Unicode strings (up to 2 GB).
 - Example: NVARCHAR(MAX)

3. Date and Time Data Types

These data types store date and time-related information.

- **DATE**: Stores only the date (year, month, day).
 - Example: DATE
- **TIME**: Stores only the time (hour, minute, second, fractions of a second).
 - Example: TIME(3) stores time with up to 3 digits of precision for fractional seconds.
- **DATETIME**: Stores date and time information with a precision of 3 milliseconds.
 - Example: DATETIME
- **DATETIME2**: Stores date and time with higher precision (up to 100 nanoseconds).

- Example: DATETIME2(7) (precision up to 7 digits).
- **SMALLDATETIME**: Stores date and time information with lower precision (minute level, with 1-minute accuracy).
 - Example: SMALLDATETIME
- **DATETIMEOFFSET**: Stores date and time with a time zone offset.
 - Example: DATETIMEOFFSET(7)

4. Binary Data Types

These data types are used to store binary data such as images, files, or encrypted data.

- **BINARY(n)**: Stores fixed-length binary data. n specifies the number of bytes (1 to 8,000).
 - Example: BINARY(10) stores exactly 10 bytes.
- **VARBINARY(n)**: Stores variable-length binary data. n specifies the maximum number of bytes (up to 8,000).
 - Example: VARBINARY(100) stores up to 100 bytes.
- **IMAGE**: Stores large binary data, such as images or files. Deprecated, use VARBINARY(MAX) instead.
 - Example: IMAGE

5. Other Data Types

These data types serve various purposes in SQL Server.

- **BIT**: Stores Boolean values, 1 for true, 0 for false.
 - Example: BIT
- **UNIQUEIDENTIFIER**: Stores globally unique identifiers (GUIDs).
 - Example: UNIQUEIDENTIFIER
- **MONEY**: Stores currency values with four decimal places.
 - Example: MONEY
- **SMALLMONEY**: Stores smaller currency values with four decimal places.
 - Example: SMALLMONEY
- **XML**: Stores XML data.
 - Example: XML

- **CURSOR:** Stores a reference to a cursor, which is used to iterate through result sets row by row.
 - Example: CURSOR
- **SQL_VARIANT:** Stores any data type except TEXT, NTEXT, and IMAGE.
 - Example: SQL_VARIANT
- **TABLE:** Stores a result set for use in variables or table-valued parameters.
 - Example: TABLE

6. Special Data Types

These are used for specific operations and functionality.

- **JSON:** SQL Server 2016+ can store JSON-formatted strings in VARCHAR, NVARCHAR, or TEXT. SQL Server does not have a specific JSON data type, but you can use string data types to store JSON content.
 - Example: NVARCHAR(MAX) for storing JSON data.
- **HIERARCHYID:** Stores hierarchical data (like organizational structures or directory paths).
 - Example: HIERARCHYID
- **GEOGRAPHY:** Stores geographic data, such as latitude and longitude.
 - Example: GEOGRAPHY
- **GEOMETRY:** Stores geometric data (for spatial data types, such as points, lines, and polygons).
 - Example: GEOMETRY

Data Model:**1. Conceptual Data Model:**

The **conceptual model** is a high-level representation of the data requirements of the business or system. It is not concerned with how the data will be stored or accessed in the database system. Instead, it focuses on **what data is needed** and **how entities and their relationships** are defined.

- **Purpose:** To provide a clear understanding of what data is important for the system and to ensure all stakeholders agree on the scope of the data. It is designed to be independent of any specific database management system (DBMS).
- **Main Features:**
 - Defines **entities** (e.g., Customer, Order, Product).
 - Describes the **relationships** between entities (e.g., a Customer places an Order).
 - Includes **attributes** (e.g., Customer has a name, email, and phone number).
 - Often represented using an **Entity-Relationship Diagram (ERD)**.

Example of a Conceptual Model (ERD):

- **Entities:** Customer, Order, Product
- **Relationships:** A Customer places an Order, an Order contains Products
- **Attributes:** Customer (name, address), Order (date, total_amount), Product (name, price)

2. Logical Data Model:

The **logical model** takes the conceptual model and translates it into a more detailed structure that is independent of any specific DBMS but is closer to the actual database structure. It defines **how the data will be organized** logically and how the entities and relationships will be implemented.

- **Purpose:** To refine the data model by specifying data types, constraints, and keys. The logical model is still platform-independent but makes the design more detailed and suitable for implementation in a specific database system.
- **Main Features:**

- Defines **tables** (e.g., Customer, Order, Product).
- Specifies **primary keys** for uniqueness (e.g., CustomerID for the Customer table).
- Defines **foreign keys** for relationships between tables (e.g., CustomerID in the Order table references the Customer table).
- Specifies **attributes' data types** (e.g., VARCHAR for name, INT for ID, DECIMAL for price).
- Still independent of any physical implementation considerations.

Example of a Logical Model:

- **Tables:** Customer, Order, Product
- **Columns:**
 - Customer table: CustomerID (Primary Key), Name, Email
 - Order table: OrderID (Primary Key), CustomerID (Foreign Key), OrderDate, TotalAmount
 - Product table: ProductID (Primary Key), ProductName, Price
- **Relationships:** The Order table has a foreign key to Customer, and the Order table references Product with an intermediary table (OrderDetails).

3. Physical Data Model:

The **physical model** defines **how the data will be stored** and accessed within a specific DBMS. This is where the implementation details are specified, such as indexes, partitioning, storage locations, and performance optimization techniques. The physical model takes into account the performance, hardware, and DBMS-specific features.

- **Purpose:** To implement the logical model in a specific database system, ensuring that the design is optimized for performance, storage, and retrieval.
- **Main Features:**
 - Defines **tables** and their **storage** structures (e.g., heap tables, clustered tables).
 - Specifies **indexes** for faster searching (e.g., an index on CustomerID).
 - Details **partitioning** schemes if necessary (e.g., partitioning the Order table by year).
 - Defines **denormalization** (if performance considerations require it).

- Takes into account the DBMS-specific features like tablespaces, file organization, and data access methods.

Example of a Physical Model:

- **Tables:** Customer, Order, Product
- **Indexes:**
 - An index on CustomerID to speed up lookups.
 - A composite index on OrderDate and CustomerID to improve query performance.
- **Storage Considerations:**
 - Use of tablespaces to organize large datasets.
 - Considerations for storing Order data in partitioned tables by year for faster querying.
 - Column data types optimized for the DBMS (e.g., VARCHAR(100) instead of TEXT).

Summary of Differences:

Aspect	Conceptual Data Model	Logical Data Model	Physical Data Model
Focus	What data is needed and how entities relate	How data is organized logically, independent of DBMS	How data is actually stored and accessed in a DBMS
Level of Detail	High-level, abstract	More detailed but platform-independent	Specific to DBMS, including performance considerations
Components	Entities, relationships, attributes	Tables, keys, relationships, data types	Tables, indexes, partitions, file organization
Representation	Entity-Relationship Diagram (ERD)	Relational model (tables, keys)	Database schema with specific optimizations
Example	Customer, Order, Product entities with relationships	Tables for Customer, Order, Product with keys	Indexes on CustomerID, partitioning of Order table

Conclusion:

- **Conceptual Model:** Focuses on understanding the business data and relationships.
- **Logical Model:** Defines how that data should be structured in a relational manner, independent of any specific DBMS.
- **Physical Model:** Implements the data structure in a specific database system with considerations for performance, storage, and access.

The **conceptual model** is abstract and helps in gathering requirements, the **logical model** refines the design, and the **physical model** is the actual implementation that is optimized for the target system.

1/17/2025	12PM - 1 PM
-----------	-------------

DDL (Data Definition Language)

DDL (Data Definition Language) in SQL refers to the set of SQL commands used to define and manage the structure of a database, such as creating, altering, or deleting tables and schemas. The main DDL commands are:

1. **CREATE:** Used to create a new database object like a table, view, or index.
2. **ALTER:** Used to modify an existing database object, such as a table structure.
3. **DROP:** Used to delete a database object.
4. **TRUNCATE:** Used to remove all records from a table but retains the table structure.
5. **RENAME:** Used to rename an existing database object.

Examples of DDL Commands:

1. **CREATE TABLE:** This command is used to create a new table in the database.

```
CREATE TABLE Employees (
```

```
    ID INT PRIMARY KEY,
```

```
    Name VARCHAR(100),
```

```
    Age INT,
```

```
    Department VARCHAR(50)
```

```
);
```

2. **ALTER TABLE:** This command is used to modify an existing table, such as adding a new column.

```
sql
```

```
-----
```

```
ALTER TABLE Employees
```

```
ADD Email VARCHAR(100);
```

3. **DROP TABLE:** This command is used to delete an existing table and all its data from the database.

```
sql
```

```
-----
```

```
DROP TABLE Employees;
```

4. **TRUNCATE TABLE:** This command is used to remove all records from a table, but unlike DROP, it does not remove the table structure.

```
sql
```

```
-----
```

```
TRUNCATE TABLE Employees;
```

5. **RENAME:** This command is used to rename an existing database object, such as a table.

```
sp_rename 'dbo.DeadB', 'DeadC'
```

```
sp_rename '[dbo].[DeadC].[age]', 'CustomerAge'
```

Notes:

- DDL commands typically involve changes to the database schema and structure, and they are often irreversible.
- DDL commands are auto-committed in most databases, meaning once executed, the changes are permanent and cannot be rolled back.
- It's important to use these commands carefully, especially when dropping or truncating tables, as they can lead to the loss of data.

DCL

In SQL, **DCL (Data Control Language)** is a subset of SQL commands used to control access to data stored in a database. DCL primarily deals with granting and revoking permissions or access rights to users and roles within a database system. The main DCL commands are:

1. GRANT**2. REVOKE**

These commands are used to control who has the ability to perform certain operations (like SELECT, INSERT, UPDATE, DELETE, etc.) on specific database objects (tables, views, procedures, etc.).

1. GRANT Command

The **GRANT** command is used to provide specific privileges to users or roles. This is how you allow a user or a role to access a database object and perform operations on it.

Syntax:

sql

GRANT privilege_type ON object TO user_or_role;

- **privilege_type:** The type of permission (such as SELECT, INSERT, UPDATE, DELETE, ALL, etc.).
- **object:** The name of the database object (e.g., table, view, stored procedure, etc.).
- **user_or_role:** The user or role to which the privilege is being granted.

Example:

Granting a user SELECT permission on a table:

sql

GRANT SELECT ON Employees TO User1;

This statement allows User1 to select (read) data from the Employees table.

Granting multiple permissions:

sql

GRANT SELECT, INSERT, UPDATE ON Employees TO User1;

This statement grants User1 the ability to SELECT, INSERT, and UPDATE data in the Employees table.

2. REVOKE Command

The REVOKE command is used to remove previously granted privileges from a user or role. This is how you deny or withdraw specific permissions from users or roles.

Syntax:

sql

REVOKE privilege_type ON object FROM user_or_role;

- **privilege_type:** The type of permission being revoked (e.g., SELECT, INSERT, UPDATE, etc.).
- **object:** The database object (e.g., table, view, etc.).
- **user_or_role:** The user or role from whom the privilege is being revoked.

Example:

Revoking the SELECT permission from a user:

sql

REVOKE SELECT ON Employees FROM User1;

This statement removes the SELECT permission from User1 on the Employees table.

Revoking multiple permissions:

sql

REVOKE SELECT, INSERT ON Employees FROM User1;

This statement removes both the **SELECT** and **INSERT** permissions from **User1** on the **Employees** table.

Example Use Cases for DCL Commands

1. Granting Permissions to a User:

sql

-- Grant SELECT and INSERT permissions on a table

GRANT SELECT, INSERT ON Orders TO User2;

2. Revoking Permissions from a User:

sql

-- Revoke DELETE permission from a user

REVOKE DELETE ON Orders FROM User2;

3. Granting Permissions to a Role:

sql

-- Grant SELECT and UPDATE permissions to a role

GRANT SELECT, UPDATE ON Customers TO SalesRole;

Permissions (Privileges) in SQL

Common privileges include:

- **SELECT:** Allows reading data from a table or view.
- **INSERT:** Allows inserting new records into a table.
- **UPDATE:** Allows updating existing records in a table.
- **DELETE:** Allows deleting records from a table.

- **EXECUTE:** Allows executing a stored procedure or function.
- **ALTER:** Allows modifying the structure of a table or other objects (like changing columns).
- **REFERENCES:** Allows creating foreign key relationships that reference a table.
- **ALL:** Grants all available permissions (can be used for convenience).

Key Points About DCL:

- **Scope: GRANT and REVOKE commands can be applied to individual users or roles.**

A **login** is at the **SQL Server instance level** (allows access to the server).

```
CREATE LOGIN TestLogin WITH PASSWORD = 'StrongP@ssword123!';
```

if you want to use a Windows user/group:

```
CREATE LOGIN [DOMAIN\UserName] FROM WINDOWS;
```

A **user** is inside a database and is mapped to a login.

```
USE YourDatabase;
```

```
CREATE USER TestUser FOR LOGIN TestLogin;
```

Roles are used to group permissions.

```
CREATE ROLE DataReaders;
```

Add User to Role

```
ALTER ROLE DataReaders ADD MEMBER TestUser;
```

Grant Permissions to Role

```
GRANT INSERT, UPDATE, DELETE ON SCHEMA :: dbo TO DataReaders;
```

 Summary Workflow:

1. **Create Login** (server level) → CREATE LOGIN
2. **Create User** (database level) → CREATE USER
3. **Create Role** (database level) → CREATE ROLE
4. **Grant permissions to Role** → GRANT
5. **Add User to Role** → ALTER ROLE ADD MEMBER

- **Security:** DCL commands are essential for database security and controlling access. They are typically managed by database administrators (DBAs).
- **Role-Based Access:** Instead of assigning permissions to individual users, permissions can be granted to roles (groups of users), and then users can be assigned to roles. This simplifies permission management.

Conclusion

DCL commands (GRANT and REVOKE) in SQL Server are essential for controlling user access and security at the database level. They allow database administrators to manage which users or roles have permission to perform certain operations on database objects. Properly using these commands helps protect sensitive data and ensures that users only have access to the data they need to perform their tasks.

1/17/2025	2 PM - 3 PM
-----------	-------------

DML (Data Manipulation Language)

DML (Data Manipulation Language) in SQL refers to the set of commands used to manipulate the data in the database. These commands allow you to insert, update, delete, and retrieve data from database tables. The main DML commands are:

1. **INSERT:** Adds new records to a table.
2. **UPDATE:** Modifies existing records in a table.
3. **DELETE:** Removes records from a table.
4. **SELECT:** Retrieves data from one or more tables (though it's technically not a manipulation of data, it is still part of DML).

Examples of DML Commands:

1. **INSERT INTO:** This command is used to insert new rows of data into a table.

sql

```
INSERT INTO Employees (ID, Name, Age, Department)
VALUES (1, 'John Doe', 30, 'HR');
```

You can also insert multiple rows at once:

```
sql
```

```
-----
```

```
INSERT INTO Employees (ID, Name, Age, Department)
VALUES
(2, 'Jane Smith', 25, 'Marketing'),
(3, 'Alice Johnson', 28, 'Finance');
```

2. **UPDATE:** This command is used to update existing records in a table.

```
sql
```

```
-----
```

```
UPDATE Employees
SET Age = 31
WHERE ID = 1;
```

In this example, the Age of the employee with ID = 1 is updated to 31.

3. **DELETE:** This command is used to delete records from a table.

```
sql
```

```
-----
```

```
DELETE FROM Employees
WHERE ID = 1;
```

This will delete the record of the employee with ID = 1. Be careful with the DELETE command; without a WHERE clause, it will delete all rows from the table.

4. **SELECT:** This command retrieves data from one or more tables.

```
sql
```

```
-----
```

```
SELECT * FROM Employees;
```

The * is used to select all columns. You can also specify specific columns:

sql

SELECT Name, Department FROM Employees;

You can also apply conditions with the WHERE clause to filter data:

sql

SELECT Name, Age FROM Employees

WHERE Department = 'HR';

Notes:

- DML commands affect the actual data in the database, unlike DDL commands which affect the structure.
- Unlike DDL commands, DML commands **do not** alter the schema of the database and are generally **not auto-committed** (depending on the database settings, changes can be rolled back).
- Always be careful when using UPDATE and DELETE commands, as they can permanently alter or remove data.

1/17/2025	3 PM - 4 PM
-----------	-------------

Practice Session

1/17/2025	4 PM - 5 PM	Practice Session
1/17/2025	5 PM - 6 PM	: HR Training
1/17/2025	6 PM - 7 PM	: HR Training

